Appendix A

GLAM: Geant4 Language Assisted Modeling

The application of Monte Carlo simulations in the fields of particle and radiation physics frequently necessitates specialized expertise in simulation frameworks and programming languages. Geant4 is a powerful and flexible environment for simulating particle transport and interactions with matter, but even relatively simple simulation setups require familiarity with its software structure, programming interfaces, and configuration of components such as geometry, materials, particle sources, physics processes, and scoring. For users who need to perform only a limited number of relatively simple simulations, acquiring this knowledge can represent a significant effort compared with the actual complexity of the problem.

This appendix presents GLAM[1] (Geant4 Language-Assisted Modeling), a prototype framework developed to investigate the use of Large Language Models (LLMs) to assist in the construction of Geant4 simulations. The objective is to enable users to describe a simulation setup using natural language and automatically translate it into a structured representation that can subsequently be converted into executable Geant4 code. This intermediate approach was selected to prevent the LLM from directly generating arbitrary Geant4 source code. While LLMs can generate syntactically valid code, the unconstrained nature of the generated code can complicate validation, reproduction, and control of the resulting simulations. A structured intermediate representation (IR) allows the possible configurations to be explicitly defined and constrained, while separating the interpretation of the user's request from its implementation in Geant4. The LLM is responsible for interpreting the user's intent, while the translation into Geant4 code is handled deterministically, allowing the generated configuration to be checked before execution.

GLAM should be regarded as a prototype rather than a complete and general-purpose simulation framework. A preliminary functional version has been developed and tested, demonstrating the feasibility of the proposed approach. However, the current implementation is still limited in terms of the range of simulation configurations that it can support.

[1] This project was developed as a personal exploration outside the scope of the PhD research activities, with the aim of informally investigating the feasibility of the proposed approach.

A.1 Overview of the GLAM framework

The workflow begins with a natural-language description of the desired simulation. The user does not need to specify the syntax of Geant4 classes or the structure of the C++ application. Instead, the user describes the relevant physical and geometrical elements at a higher level, by writing a prompt in natural language. Depending on the requested simulation, this may include the geometry of the detector, the materials used and the properties of the primary particle source. The LLM interprets this description and converts it into the structured representation defined by GLAM.

The generated representation constitutes the central interface of the framework. Rather than allowing the LLM to directly modify the Geant4 source code, GLAM defines a set of supported simulation components and represents them through a dedicated IR. This approach restricts the output of the LLM to a predefined structure and makes the generated configuration independent of the specific implementation details of the Geant4 application. The IR can therefore be inspected and validated before any Geant4 code is generated. Once the IR has been generated, it is processed by a deterministic validation stage. The validator verifies the structure of the representation and ensures that the requested elements are compatible with the capabilities and constraints defined by GLAM. This stage is of particular importance because a syntactically valid LLM output does not necessarily correspond to a valid GLAM configuration. The validator verifies that the generated representation aligns with the structure and constraints defined by the framework before translating it into Geant4 code. The validated IR is then translated into the corresponding Geant4 C++ implementation. A compiler maps the high-level elements described in the IR to predefined functions and components of the Geant4 application. The generated application can then be compiled and executed using the Geant4 environment. The resulting data are stored in a structured output file, allowing the simulation results to be further inspected or analyzed independently of the language model. The overall workflow can therefore be summarized as

$$\underbrace{\text{Natural-language request} \rightarrow \text{LLM} \rightarrow \text{IR}}_{\text{Language-based interpretation}} \longrightarrow \underbrace{\text{Validation} \rightarrow \text{Geant4 code generation}}_{\text{Deterministic processing}}$$

This organization introduces a clear separation between the probabilistic component of the framework, represented by the LLM, and the deterministic components responsible for validation, code generation, and execution. The LLM is used to interpret the user's intent and express it through the available IR elements, whereas the subsequent stages operate according to explicitly defined rules. This architecture therefore provides a more controlled alternative to unrestricted LLM-based code generation while retaining the flexibility of natural-language interaction. A natural consequence of the IR-based approach is that the LLM cannot accommodate every possible user request. Its output is constrained by the elements and configurations defined by the IR, so only simulation setups that can be expressed through the supported geometries, operations, source configurations, and scoring options can be generated. This restriction represents both a strength and a limitation of the framework: it prevents the LLM from freely generating unsupported Geant4 configurations, providing greater control over the resulting simulations, but at the same time limits the range of setups that can be produced. The configuration space of GLAM is therefore necessarily a subset of the full configuration space offered by Geant4. The following sections describe the main stages of the GLAM workflow in more detail.

A.1.1 LLM-based generation

The first stage of the GLAM workflow is the generation of the IR from the user's natural-language request. In the current implementation, GLAM does not rely on a language model developed or trained specifically for the framework. Instead, it interacts with an existing LLM through an Application Programming Interface (API). An API provides a programmatic interface through which an external software application can send a request to a service and receive its response. In this scenario, the GLAM backend transmits the user's request to an externally hosted LLM. In return, the LLM provides the generated IR as a response. The current implementation was developed using Google's Gemini models [76], although the general architecture is not intrinsically tied to a specific model.

The LLM is not provided with only the user's request. To generate an appropriate IR, the prompt also contains a set of instructions and examples defining how simulation descriptions should be translated into the GLAM representation. These elements form the knowledge base used during prompt construction. The instructions explicitly define the expected output format and constrain the model's behavior. For instance, the model is required to return only a valid JSON object, without explanations or Markdown formatting, use only predefined structures and elements, and avoid inventing unspecified information or introducing Geant4 implementation details. Numerical values must be preserved as numerical JSON values, while quantities such as lengths and energies must use the units supported by the framework. The instructions also specify relevant geometric conventions, such as the use of half-lengths for primitive solids. These constraints minimize ambiguities and ensure that the LLM operates within the predefined configuration space of GLAM. The knowledge base also contains a collection of 25 representative examples. These examples illustrate valid uses of the framework's various components and provide the LLM with concrete mappings between high-level simulation requirements and their corresponding IR representation. The examples encompass individual aspects such as geometry, particle sources, and scoring of the deposited energy, as well as composed configurations involving multiple components. They therefore complement the general instructions by showing the model how the defined representation is expected to be used in practice.

A.1.2 Intermediate representation

The IR is the interface between the LLM and the deterministic components of GLAM. It provides a structured description of the simulation that can be processed independently of the language model and subsequently translated into Geant4 code. In the current implementation, the IR is represented as a JSON object divided into three main sections: `geometry`, `source`, and `scoring`. These sections respectively contain the information required to define the geometry of the simulation, the primary particle source, and the sensitive volume to be recorded during the simulation.

The geometry section allows the simulation setup to be described in terms of a predefined set of geometrical primitives and operations. Primitive solids currently include boxes, cylinders, spheres, and cones, while boolean operations allow these objects to be combined through unions, subtractions, and intersections. Each object is identified by a unique name, and its relevant properties, such as dimensions, material, and position, are explicitly specified in the IR. The representation enables objects to be placed individually or according to predefined stacking configurations, with the corresponding positions and volume relationships explicitly specified in the IR. The source section describes the primary particle used in the simulation. It specifies the particle type, energy, position, and spatial or angular distribution of the source. The current implementation supports

point, disk, and spherical source distributions, together with directional or isotropic emission. The scoring section specifies the volumes for which energy deposition is recorded during the simulation. Example A.1 shows the JSON IR generated by GLAM in response to the following user prompt:

> Create a simple Geant4 simulation with a silicon cylindrical target of 5 cm radius and 10 cm height, placed at the center of the world volume. Use a 5 MeV gamma-ray point source located at (0, 0, -20 cm), emitting in the positive z direction. Record the energy deposited in the silicon target.


```
{
  "geometry": {
    "primitives": {
      "cylinder": [
        {
          "name": "Target",
          "r_min": {
            "value": 0.0,
            "unit": "cm"
          },
          "r_max": {
            "value": 5.0,
            "unit": "cm"
          },
          "half_height": {
            "value": 5.0,
            "unit": "cm"
          },
          "material": "G4_Si"
        }
      ]
    },
    "placement": [
      {
        "type": "place",
        "child": "Target",
        "mother": "worldLV",
        "position": [
          {
            "value": 0.0,
            "unit": "cm"
          },
          {
            "value": 0.0,
            "unit": "cm"
          },
          {
            "value": 0.0,
            "unit": "cm"
          }
        ],
```

```
      "name": "TargetPV"
    }
  ]
 },
 "source": {
  "particle": "gamma",
  "energy": {
    "value": 5.0,
    "unit": "MeV"
  },
  "type": "point",
  "position": [
    {
      "value": 0.0,
      "unit": "cm"
    },
    {
      "value": 0.0,
      "unit": "cm"
    },
    {
      "value": -20.0,
      "unit": "cm"
    }
  ],
  "direction": [
    0.0,
    0.0,
    1.0
  ]
 },
 "scoring": [
  "Target"
 ]
}
```

Listing A.1: Example of an Intermediate Representation generated from a natural-language simulation request.

A.1.3 Validator

Before being passed to the compiler, the generated IR is processed by a deterministic validation stage. The purpose of this stage is to verify that the representation is structurally consistent and complies with the configuration rules defined by GLAM. As the IR is generated by an LLM, this step is required to ensure that the output conforms to the predefined interface before any Geant4 code is generated.

The validation is based on the predefined IR schema, which serves as a template for the configurations supported by GLAM. The schema delineates the comprehensive architecture of the IR, encompassing its constituent sections, the fields available within each section, and the range of con-

figurations that can be defined. The generated JSON is then checked against a predefined structure. The validator verifies that the required sections and fields are present, that the specified elements belong to the supported configurations, and that references and relationships between elements are consistent. Consequently, the LLM is constrained to populate a predefined representation rather than constructing the structure of the simulation description freely.

It is important to note that this validation stage does not attempt to establish the physical correctness of the requested simulation. Instead, it verifies that the generated representation is compatible with the formal structure and capabilities of GLAM. This distinction is significant because the validation stage provides control over the interface between the LLM and the deterministic backend, rather than replacing physical verification of the resulting simulation.

If the IR successfully passes all validation checks, it can be passed to the compiler. Otherwise, the process is interrupted and a validation error is returned, preventing an invalid representation from being translated into Geant4 code.

A.1.4 Compiler

Once the IR has been validated, it is passed to the GLAM compiler. The compiler's primary function is to translate the high-level description contained in the IR into the corresponding C++ implementation. Each element of the IR is mapped to a corresponding function or operation with a known implementation. For instance, geometrical primitives are translated into predefined geometry-building functions, while source parameters are converted into calls that configure the primary particle generator. When a boolean object is defined using other geometrical objects, the compiler ensures that these objects are created before constructing the resulting solid.

The generated code is not intended to constitute an entire Geant4 application. GLAM relies on a predefined Geant4 backend that contains the general simulation infrastructure, while the compiler generates the configuration-dependent portions of the application. These portions are inserted into predefined locations in the existing source files. Consequently, the resulting application can be compiled using the standard Geant4 toolchain. This deterministic translation process establishes a fixed relationship between the validated IR and the resulting Geant4 implementation. Once the IR has been generated and validated, the LLM is no longer involved in the construction of the simulation. The compiler applies predefined translation rules to the representation, ensuring that the same IR corresponds to the same implementation within the GLAM backend. At the same time, the range of configurations that can be generated is limited by the capabilities encoded in the IR and implemented by the compiler. Extending GLAM to support additional Geant4 functionality therefore requires corresponding extensions to both the representation and the deterministic compilation logic.

A.2 Prototype validation

The GLAM prototype was validated in two complementary phases, designed to separately assess the deterministic part of the framework and the complete natural-language-to-simulation workflow. In the first phase, a set of IR configurations was constructed manually and used as input to the deterministic components of GLAM. As these configurations do not involve the LLM, this test isolates the subsequent stages of the pipeline and verifies that a valid IR is correctly translated into the corresponding Geant4 configuration, compiled, and executed. This approach enables a direct assessment of the relationship between the IR and the resulting simulation, independent of the

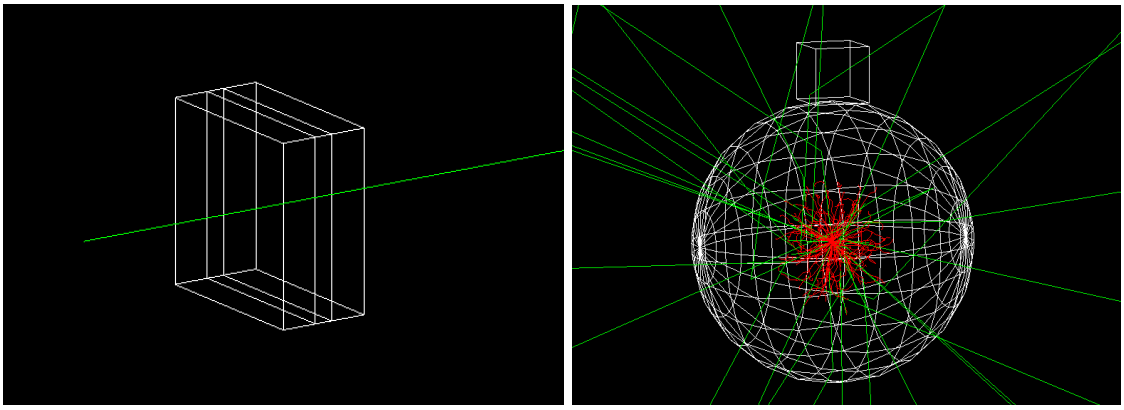


Figure A.1: Geometries generated by GLAM from the two natural-language prompts. Left: geometry generated from the first prompt. Right: geometry generated from the second prompt.

language-based generation stage. This provides a useful separation between errors introduced by the LLM and errors in the deterministic backend.

For the deterministic part of the framework, a validation set comprising five configurations was designed to evaluate various aspects of the supported IR. These include a simple silicon box, a configuration containing stacked volumes, an isotropic particle source, a geometry involving a cylindrical subtraction, and a configuration testing the selection of volumes for scoring. The selected cases therefore cover primitive geometry, relationships between volumes, boolean geometry operations, source configurations, and scoring.

In the second phase, the complete GLAM workflow was tested starting from natural-language simulation requests. In this scenario, the LLM was responsible for generating the IR from the user's prompt. The generated representation was then passed through the internal validation stage. If valid, it was translated into Geant4 code and executed. This test evaluates the entire chain, from the interpretation of the user's request to the resulting simulation. As representative examples, the following natural-language requests were provided to the system:

> Create a simple Geant4 detector consisting of three stacked layers along the z axis: a 10 cm × 10 cm × 2 cm silicon layer, a 10 cm × 10 cm × 1 cm lead layer, and another 10 cm × 10 cm × 2 cm silicon layer. Place the layers with their centers separated by 1.5 cm along the z axis. Use a 5 MeV gamma-ray point source located at (0, 0, -10 cm), emitting in the positive z direction. Record the energy deposited in all three layers.

> Create a simple Geant4 simulation with a spherical water volume of radius 5 cm at the center of the world. Place a silicon box above the sphere, with the centers of the two volumes separated by 6 cm. Use a 5 MeV isotropic electron point source at the center of the sphere. Record the energy deposited only in the silicon box.

Figure A.1 shows the geometries produced by the GLAM pipeline in response to the two prompts.

The validation demonstrates the functional operation of the current prototype within its supported configuration space. The tests confirms that the requested configurations can be represented by the IR, that they satisfy the structural constraints imposed by the validator, and that they can

be translated and executed through the predefined Geant4 backend. It should be noted that the tests performed here are not intended to demonstrate that GLAM can reproduce arbitrary Geant4 applications. The current IR and compiler only expose a subset of the full functionality available in Geant4. The successful execution of the validation examples should therefore be interpreted as evidence that the proposed architecture is technically feasible for the selected class of relatively simple simulations, rather than as a demonstration of general-purpose Geant4 generation.

A.3 Limitations and future extensions

GLAM should be regarded as a proof of concept and a prototype rather than as a complete alternative to Geant4. The current implementation was developed primarily to investigate the feasibility of using an LLM to translate natural-language descriptions into constrained and executable Geant4 configurations. Consequently, the supported geometries, source configurations, and scoring options are intentionally limited. These restrictions reflect the current stage of development rather than intrinsic limitations of the approach, and the framework can be progressively extended by introducing additional elements into the IR and corresponding validation and compilation rules. However, although the representation can be extended to support increasingly complex configurations, no finite predefined representation can cover every possible geometry or simulation that a user may wish to construct. A distinction will therefore always remain between the configurations supported by GLAM and the full flexibility offered by Geant4. This limitation is consistent with the intended scope of the project. GLAM is not intended to replace Geant4 or provide a natural-language interface to every possible simulation. Rather, the objective is to progressively cover a broad class of relatively simple simulation setups for which the full flexibility of Geant4 is not required. Within this scope, natural-language interaction can reduce the amount of Geant4-specific knowledge required and make the framework accessible to users without extensive Geant4 expertise.

Another constraint is inherent to the LLM nature itself, which still plays a probabilistic role in the workflow. Despite the constraints imposed by the IR and validator, the generated representation may not correctly interpret ambiguous or underspecified requests. The validator can detect structural inconsistencies, but cannot determine whether the generated configuration accurately reflects the user's physical intent. The current prototype also lacks an automatic correction loop for iteratively modifying failed configurations. Such a mechanism could improve usability, although it would require additional safeguards to ensure that generated modifications remain consistent with the predefined IR.

One potential expansion of the project would be to replace or supplement the text-based LLM with a multimodal model that can process images. In this scenario, the input to GLAM would not necessarily be limited to a natural-language description. A simple drawing, schematic, sketch, or photograph of a physical setup could be provided as input and interpreted by the model to construct the corresponding IR. The deterministic validation and compilation stages could then remain unchanged, preserving the separation between the interpretation of the input and the generation of Geant4 code. This would further simplify the definition of simulations, allowing users to describe a setup not only through text but also through a visual representation. In its most advanced form, GLAM could also be offered as a cloud-based service. This would provide a conversational interface that allows users to describe the desired simulation in natural language. The corresponding Geant4 simulation would then be generated without direct interaction with the underlying code. This implementation has the potential to transform GLAM from a local prototype into an accessible simulation platform, where the construction and execution of Monte Carlo simulations become as

straightforward as asking a question and receiving a corresponding result.

Overall, the prototype demonstrates the feasibility of the proposed approach: a natural-language description can be converted into a constrained intermediate representation, validated independently of the language model, deterministically translated into Geant4 code, and executed within a predefined simulation backend. Future development will focus on expanding the supported configuration space, increasing the validation set, and improving the robustness of the interaction with the LLM. The objective is not to reproduce the full generality of Geant4, but to provide a sufficiently broad range of relatively simple simulations that can be constructed through natural language, lowering the entry barrier for users without extensive Geant4 expertise.